Heterogeneous-Resource-Aware Federated Learning with Intelligent LoRA Allocation and Aggregation

Youye Xie, Yao Lian, Kevin Chen, Abdul Latif, Lingzhi Zhao, and Reza Farivar
The Grainger College of Engineering, University of Illinois Urbana-Champaign, Urbana, IL, USA
{youyex2, yaolian, kevinc17, alatif5, lz26, farivar2}@illinois.edu

***Abstract*—Federated Learning (FL) enables privacy-preserving model training across distributed devices, making it ideal for edge and cloud computing environments. Low-Rank Adaptation (LoRA), a parameter-efficient fine-tuning technique, has been widely integrated into FL to reduce computational and communication overhead. However, heterogeneous client memory and network constraints, which are ubiquitous in real-world applications, pose significant challenges to optimizing LoRA rank utilization and achieving coherent aggregation of updates from resource-diverse clients. In this paper, we introduce HRALoRA, a Heterogeneous-Resource-Aware LoRA-based federated learning, to dynamically estimate a client's maximum LoRA rank budget based on actual resource limitations, encompassing both memory and network bottlenecks in heterogeneous environments. After obtaining the rank budget, HRALoRA employs an intelligent rank allocation algorithm that adaptively assigns ranks to different layers of models based on the layer importance measured by the Fisher Information Matrix (FIM) score, ensuring efficient participation in FL. Moreover, we propose an alternating training strategy to improve robustness under heterogeneous data distributions while reducing communication overhead, and design an SVD-based aggregation method to effectively aggregate heterogeneous LoRA updates.**

Experimental results show that HRALoRA attains performance on par with or exceeding that of prior LoRA-based FL approaches, despite using only half the trainable parameters. Our approach enhances scalability and efficiency in cluster and cloud-based FL deployments, paving the way for practical adoption in resource-constrained distributed systems.

***Index Terms*—Federated Learning, Low-Rank Adaptation, Heterogeneous clients, Parameter-efficient fine-tuning.**

I. Introduction

A. Challenges in Federated Learning

Motivated by the distributed nature of data and growing privacy concerns, federated learning (FL) has emerged as a paradigm for collaborative model training across distributed clients while preserving data privacy [1]. Comprehensive surveys highlight its applications and challenges, including data privacy, communication efficiency, and handling heterogeneity in client data, devices, and networks [2], [3]. In federated learning, a central server maintains a global model, while each edge device keeps a local copy. Edge devices train locally on private data, sending only model updates (e.g., gradients or weights) to the server. The server aggregates these to update and redistribute the global model. This iterative process enables collaborative training while keeping raw data private. However, federated learning faces three major challenges:

- High communication overhead between server and clients due to the large size of the training model.
- Non Independent and Identically Distributed (IID) data across clients, hindering global model convergence [4].
- Heterogeneous system resources among edge devices, making aggregation and uniform training difficult.

B. Low-rank Adaptation for Parameter-efficient Fine Tuning

Due to ever-increasing model sizes and resource-constrained devices, the parameter-efficient fine-tuning (PEFT) [5], [6] has been extensively studied. The goal of PEFT is to fine-tune a big model, but utilizing only a small portion of parameters for effective tuning and reduced communication overhead. Among PEFT methods in the literature [7]–[9], the Low-rank Adaptation (LoRA) [10] has attracted much attention due to its superior fine-tuning performance and significantly low number of trainable parameters. By exploiting the latent low-rank characteristic of the model weights, experimental results demonstrate that LoRA enables effective fine-tuning of large language models (LLMs) by training less than 1% of the total model parameters. [10], [11].

Starting from a pre-trained large backbone model, LoRA freezes the base parameters, and only trains the attached low-rank adaptation modules. A LoRA module takes the form [10]

$$W = W_0 + BA \in R^{n\times m} \tag{1}$$

where $W_0 \in R^{n\times m}$ is the model's original weight, and $B \in R^{n\times r}$ and $A \in R^{r\times m}$ are the LoRA module matrices. r is the rank of LoRA. Due to the small rank of $r << \min(n, m)$, the LoRA module only contains a small number of weights compared to W_0. However, majority of prior Federated LoRA methods [11], [12] assume the same LoRA rank among clients, which overlooks the substantial differences in system resources across clients. Moreover, classical LoRA averaging update leads to degraded performance with non-IID data [4].

C. Heterogeneous-Resource-Aware Federated Learning

In this paper, we focus on tackling the aforementioned federated learning challenges for PEFT using LoRA. We propose the Heterogeneous-Resource-Aware LoRA-based Federated Learning (HRALoRA), which estimates each client's LoRA rank budget based on the client's memory and bandwidth resources and effectively utilizes the LoRA budget to train a well-performing global model. We focus on memory and network because they bound feasibility and communication. Compute heterogeneity (FLOPs/TOPS/HBM bandwidth) affects local update time and straggling and can be handled

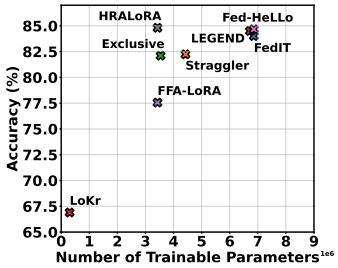


Fig. 1. Fine-tuning performance on IID CIFAR100 [13] using ViT model *facebook/deit-small-patch16-224* [14]. Using the same LoRA rank budget, HRALoRA achieves the highest accuracy among all compared approaches, including LoKr [15], FFA-LoRA [16], LEGEND [17], FedIT [12], Fed-HeLLo [18], Exclusive, and Straggler. Notably, HRALoRA attains this performance with only a small number of trainable parameters.

orthogonally (e.g., via straggler mitigation). Evaluating that integration is future work. Given the same client-side resource constraints, and therefore the same LoRA rank budget, Fig. 1 shows an example of fine-tuning performance on CIFAR100 [13], where HRALoRA outperforms other methods using a smaller number of trainable parameters. Our contributions are summarized as follows.

- To address the difficulty of training with heterogeneous clients and increase client engagement, we propose a LoRA rank budget estimation method to obtain each client's rank budget according to its memory and communication bandwidth.
- To exploit the allocated rank budget in a client, HRALoRA intelligently selects the LoRA rank distribution among model layers using Fisher Information Matrix (FIM) score [19], [20]. This is a constrained resource allocation problem and we use FIM as a sensitivity proxy and apply a greedy heuristic. As a result, a higher LoRA rank is assigned to the more important layers.
- To enhance the training robustness to non-IID data and reduce the communication overhead, HRALoRA applies local training in an alternating manner, inspired by Alternating Direction Method of Multipliers (ADMM) [21] and FFA-LoRA [16] which only trains LoRA matrix B for stable training under heterogeneous data. Because only one of the LoRA matrices is trained in each round, the trainable parameters and transmitted data are halved compared to other methods under the same LoRA budget.
- In the server, zero padding is applied for matrix aggregation and an SVD-based aggregation method is designed to balance the magnitude between LoRA A and B.

The proposed HRALoRA has the potential to significantly enhance the practical deployment of federated learning in real-world applications where edge devices vary widely in system capabilities. By reducing the communication and local training costs, this work lowers the barrier for participation in federated networks, enabling greater inclusivity of low-resource devices such as smartphones, IoT sensors, and medical wearables.

The rest of the paper is organized as follows. In Section II, we present the related work. Our proposed HRALoRA is presented in Sections III, and the experiments results are shown in Section IV. Finally, we conclude in Section V.

II. Related Work

A. Parameter Efficient Fine Tuning

One direction of PEFT focuses on smart parameter selection, where the fine-tuning is run on selected layers of the original model. BitFit [7] only trains the bias terms and classification layers. FjORD [8] proposes the ordered dropout algorithm to tailor the model width. Another branch of effort freezes the entire model and trains additional lightweight modules to adapt to new tasks. FedAdapter [22] trains adapters with down and up projections, and Low-Rank Adaptation (LoRA) [10] trains a bypass low-rank matrix utilizing the intrinsic low-rank characteristic of weights. There are more PEFT extension approaches which set the adapter setting automatically [23] or use a mixture of different PEFT techniques [24]. Among existing PEFT methods, LoRA has attracted much attention due to its low communication cost and stable performance compared to the full model fine-tuning [11], [25].

B. Federated PEFT with LoRA

FedIT [12] and FedPETuning [11] propose to combine LoRA and federated learning for PEFT. A simple average aggregation approach is applied following the FedAvg [1], which achieves promising results. Afterward, more techniques on LoRA have been developed for improving PEFT convergence rate and robustness. SLoRA [26] presents a 2-stage warm start method for LoRA initialization. FLoRA [27] proposes a stacking-based LoRA matrix aggregation method that supports the aggregation of LoRA modules from clients with different ranks. FFA-LoRA [16] proposes to freeze the A matrix in LoRA to mitigate the interference of noise in weight aggregation. Compared to FFA-LoRA, the proposed HRALoRA with alternating training mitigates performance degradation that occurs in FFA-LoRA when the data distribution lies outside the subspace spanned by the frozen encoder matrix A. Fedbiot [28] fine-tunes the emulator using LoRA to reduce the computation burden in clients. However, the above methods overlook heterogeneous resources in clients, which is hard to guarantee in practice.

C. Heterogeneous Client Resources

In the classical FL setting [1], each client loads and fine-tunes the global model replicated from the server. With heterogeneous clients, the training process could be elongated by clients with limited capabilities. One strategy is to identify and remove stragglers. Oort [29] proposes to exclude clients for aggregation based on their system efficiency. However, excluding clients could leave out valuable data stored in the straggler devices. Other researchers consider deploying heterogeneous models on clients with different resources. HeteroFL [30] identifies the client resource and adjusts the width of hidden channels. Depthfl [31] limits the training complexity by adjusting the trainable layer depth. Methods using model pruning for heterogeneous clients still require the

update for all local weights. Moreover, reducing the size of the server model could potentially increase the model bias [32].

To significantly reduce the communication overhead and retain the model's representation power, several LoRA-based PEFT approaches for heterogeneous clients are proposed. HetLoRA [33] proposes to apply different ranks for different clients based on their system capabilities. The aggregation of the LoRA weights with different ranks is weighted by the norm of singular values, and zero padding is applied for dimension mismatch of LoRA from different clients. Down the same path, FlexLoRA [25] averages the constructed matrices product, BA, of LoRA, in aggregation, and a singular value truncation is implemented to get the varying rank of LoRA's A and B matrices for weights redistribution. However, neither method accounts for layer-wise differences in importance and assumes a uniform LoRA rank across all layers. Compared to FlexLoRA, HRALoRA explicitly models layer-wise importance and employs SVD-based aggregation to balance the global LoRA matrices rather than redistributing ranks. In HRALoRA, each client maintains a replica of the global model including LoRA modules while activating heterogeneous LoRA ranks during training.

Recent studies have shown that accounting for layer-wise differences when inserting LoRA modules can improve fine-tuning performance. LEGEND [17] adjusts the LoRA rank budget and layer-wise rank distribution according to a client's fine-tuning time. In particular, LEGEND adopts a monotonically increasing rank assignment from the input layer to the output layer, based on the assumption that output layers have a greater impact on final performance. Compared to LEGEND, HRALoRA enables more flexible and data-driven rank allocation via Fisher Information Matrix (FIM) score [19]. Fed-HeLLo [18] assigns LoRA modules with a uniform rank to selected layers based on their importance measured by the FIM score, allowing clients with larger memory budgets to fine-tune a greater number of LoRA modules. Compared to Fed-HeLLo, HRALoRA extends FIM-based allocation to heterogeneous ranks across layers for finer-grained adaptation. Furthermore, HRALoRA introduces a resource-aware rank budget estimator and an alternating training scheme to improve robustness in the heterogeneous environment.

III. HRALoRA: Heterogeneous-Resource-Aware LoRA-based Federated Learning

An overview of the proposed HRALoRA framework is illustrated in Fig. 2. The server maintains a pre-trained model for fine-tuning and computes Fisher Information Matrix (FIM) scores on a small public proxy dataset, which provides sensitivity signals for rank/layer allocation without requiring access to clients' private data distributions [18], [34]. For the FIM scores to reliably reflect the layer sensitivity, the proxy data should be drawn from a distribution similar to that of the clients' local data. In our experiments, we use a held-out evaluation split drawn from the same distribution as the training data. In practical deployments, this proxy can be a non-sensitive dataset from the same task domain. Clients have

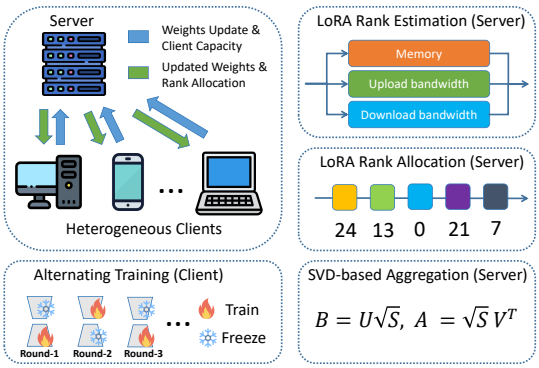


Fig. 2. The overview of HRALoRA, which consists of LoRA rank estimation, LoRA rank allocation, alternating training, and SVD-based aggregation, running in a distributed environment with heterogeneous clients.

a copy of the frozen pre-trained model. In each training round, the server collects clients' resource profiles, including memory and bandwidth capacities, along with their local LoRA weight updates. Based on the reported resource constraints, the server first determines a LoRA rank budget for each client via rank estimation. Given these budgets, a rank allocation algorithm is executed on the server to determine the layer-wise LoRA rank distribution subject to each client's rank budget. To aggregate heterogeneous LoRA updates with varying ranks, the server applies zero padding and an SVD-based aggregation scheme to fuse the client updates and produce the global LoRA weights. The updated global LoRA parameters are broadcasted to all clients for the next training round. On the client side, each client initializes its local LoRA parameters from the global model and activates a subset of LoRA modules with heterogeneous ranks according to the assigned rank distribution.

A. Rank Estimation by Memory

Majority of prominent models are based on transformer architecture [35]. The rank estimator determines the rank budget per client running a transformer model based on the equations below. Variables definition can be found in Table I.

$$m_{\text{total}} = m_{\text{par}} + m_{\text{os}} + m_{\text{fw\_b}} + m_{\text{fw\_l}} + m_{\text{grad}} + m_{\text{oh}} \tag{2}$$

$$par_{\text{lora}} = \sum_{i=1}^{n_{\text{mod}}} (m_{\text{i}} + n_{\text{i}}) \cdot r_{\text{memory}} \tag{3}$$

$$m_{\text{par}} = bytes_{\text{prec}} \cdot (par_{\text{base}} + par_{\text{lora}}) \tag{4}$$

$$m_{\text{os}} = bytes_{\text{prec}} \cdot n_{\text{s}} \cdot par_{\text{lora}} \tag{5}$$

$$m_{\text{grad}} = bytes_{\text{prec}} \cdot par_{\text{lora}} \tag{6}$$

$$m_{\text{fw\_l}} = bytes_{\text{prec}} \cdot B \cdot S \cdot (\beta_1 \cdot H + \beta_2 \cdot r_{\text{memory}} + e) \tag{7}$$

Note that the optimizer-state and gradient memory in equation (5) and (6) include only the LoRA components, as the base model is frozen during LoRA fine-tuning. Base model and framework-related variables, $m_{\text{fw\_b}}$, par_{base}, and m_{oh} can be estimated via profiling. To determine the LoRA rank r_{memory}, we first need to solve for LoRA-related variables in equation (7), β_1 and β_2, which depend on the base model architecture. The rank estimator can automatically profile

TABLE I
SUMMARY OF VARIABLES

Variable	Description
r_{memory}	Memory-constrained rank budget per client
m_{total}	Total available memory for training
m_{par}	Memory for total parameters
$m_{\text{fw\_b}}, m_{\text{fw\_l}}$	Memory for forward propagation (excluding parameters and framework overhead), base model portion and LoRA portion respectively
m_{os}	Memory for optimizer states
m_{grad}	Memory for gradients during backward pass
m_{oh}	Memory for framework overhead
$par_{\text{base/lora}}$	Parameter count of base model vs. LoRA
$bytes_{\text{prec}}$	Bytes per parameter based on precision (4 for FP32, 2 for FP16)
n_s	Number of optimizer states, 2 for Adam/Adamw [36] and 1 for SGD [37]
$n_{\text{mod}}, l_{\text{i}}$	Number of module types with added LoRA (e.g., "query" and "key" are 2 module types), and the number of layers with LoRA applied for this module type
$m_{\text{i}}, n_{\text{i}}$	Shape of added LoRA matrix for each module
B, S, H	batch size, sequence length, hidden dimension
β_1, β_2, e	coefficients β_1 and β_2, residual error e

the system and solve for their values. Specifically, the rank estimator conducts a small number of profiling runs using randomly chosen r. In each run, it profiles the forward-pass memory consumption, from which the base-model forward memory $m_{\text{fw\_b}}$ is subtracted to obtain $m_{\text{fw\_l}}$. Since B, S, H, and $bytes_{\text{prec}}$ are known at training configuration time, each profiling run yields one equation involving β_1 and β_2 as the sole unknown variables. After multiple runs, the resulting system of equations is used to solve for β_1 and β_2. After that, we combine equations (7) and (3), whose variables are known during the configuration time, and solve for the rank budget under the memory constraint r_{memory}.

B. Rank Estimation by Uploading and Downloading Speed

To maintain communication efficiency, we determine the LoRA rank r_{upload} based on a target transmission time t_{upload} (e.g., 1 second), and the expected speed s_{upload}, by solving $t_{\text{upload}} \cdot s_{\text{upload}} = bytes_{\text{prec}} \cdot par_{\text{LoRA}}$. An analogous calculation is used to derive download rank, r_{download}.

C. The Final Rank and Layer Budget

Because r_{memory}, r_{upload}, and r_{download} are independent hard constraints that must all be satisfied, the feasible set is the intersection. Thus, final total rank budget per client is the minimum of the three independent bounds to simultaneously respect the GPU memory capacity and network condition:

$$r_{\text{budget}} = \min(r_{\text{memory}},\ r_{\text{upload}},\ r_{\text{download}}). \tag{8}$$

Here r_{budget} represents the maximum LoRA rank available to a client. For instance, when $r_{\text{budget}} = 288$, the total rank across all trainable LoRA modules cannot exceed 288. Table II summarizes the rank allocation strategies adopted by different methods in the literature under a given rank budget.

TABLE II
RANK ALLOCATION STRATEGY OF LORA-BASED PEFT WITH HETEROGENEOUS CLIENTS.

Method	Rank Allocation Strategy for Each Client
FlexLoRA [25]	Uniform rank across all layers using average.
HetLoRA [33]	Uniform rank across all layers using average.
LEGEND [17]	Static rank allocation policy that biases rank assignment toward later layers of the model.
Fed-HeLLo [18]	FIM-based layer selection with a fixed LoRA rank assigned to each selected layer.
HRALoRA	FIM-based layer selection and adaptive rank allocation.

Once the overall rank budget is determined, we derive the corresponding layer budget for each client, l_{budget}. l_{budget} limits the number of layers that LoRA is trained. For the most resource-capable client, we set $l_{\text{budget}} = l_{\text{total}}$ where l_{total} is the total number of layers in the global model. Assuming that each layer is equipped with a LoRA module, the average LoRA rank per module is $r_{\text{avg}} = r_{\text{budget}}/l_{\text{budget}}$. To ensure a consistent average LoRA rank per module across clients with heterogeneous resource capacities, we scale each client's layer budget proportionally to its rank budget, i.e., the ratios of l_{budget} among clients match the ratios of their respective rank budgets. This design maintains a uniform r_{avg} across clients, leading to more balanced LoRA training and better convergence rate under heterogeneous constraints.

Having determined the per-client rank and layer budget, we propose two enhancements to federated PEFT that better exploit the heterogeneity of client resources, which will be covered in the following sections.

D. Adaptive LoRA Rank Allocation using FIM Score

Recent advances in intelligent LoRA assignment [18], [38] demonstrate that the LoRA performance could be improved by applying LoRA to layers that are more sensitive to the domain dataset. In HRALoRA, we adopt this idea into federated PEFT and extend it to enable varying and adaptive rank allocation. Specifically, the server maintains a global model with LoRA modules added in selected components for all layers, e.g., the "query" component in transformer [39]. But during local training, clients could have their activated LoRA in different layers with different ranks. The only constraint is that for a specific client, only l_{budget} layers of LoRA are activated and the total rank of all activated LoRA modules in this client equals r_{budget}, where r_{budget} and l_{budget} are determined based on this client's available memory and communication bandwidth.

In order to better exploit the heterogeneous resources, we allocate the LoRA to important layers with the number of rank increases proportional to its importance. We measure the importance or sensitivity of a layer using Fisher Information Matrix (FIM) score. During the training, FIM score is calculated for all layers, which has the form [18], [20]

$$FIM_{\text{score}}(W) = \left\| \frac{1}{N} \sum_{i=1}^{N} (\nabla_W \mathcal{L}_i \odot \nabla_W \mathcal{L}_i) \right\|_F \tag{9}$$

$\nabla_W \mathcal{L}_i$ is the gradient of the weight W using the i-th sample. N is the total number of samples. Note that we only calculate the FIM score of the components where LoRA modules are applied. A higher FIM score implies that the specific layer is more sensitive to the data, and more resources should be allocated to it. Based on the calculated FIM score, we construct a layer selection distribution following the layer selection method in Fed-HeLLo [18], and l_{budget} layers will be randomly selected based on the distribution. The layer with a higher FIM score has a higher priority to be selected.

Once the layers are selected, we extract the FIM scores of the selected layers and normalize them. As a result, we get the rank distribution indexed on the layer $P_{\text{rank,i}}$ such that $\sum_{i \in l_{\text{selected}}} P_{\text{rank,i}} = 1$. We then determine the LoRA rank in the i-th selected layer following

$$r_{\text{i}} = \min(\max(P_{\text{rank},i} \times r_{\text{budget}}, r_{\text{min}}), r_{\text{max}}), \quad i \in l_{\text{selected}} \tag{10}$$

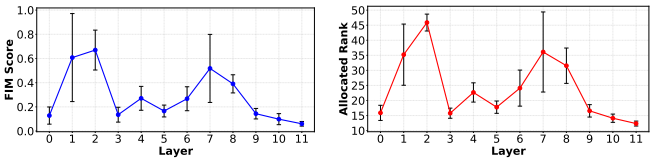
where r_{max} is the maximum allowed rank of the LoRA module and r_{min} is the minimum rank. Equation (10) ensures $r_i \in [r_{\text{min}}, r_{\text{max}}]$. To allow enough headroom for rank variation, we set $r_{\text{max}} = 2 \times r_{\text{avg}}$ and $r_{\text{min}} = r_{\text{avg}}/2$ to create a smooth rank distribution. l_{selected} is the list of selected layers for this client. In an extreme skewed case where $P_{\text{rank},i} \times r_{\text{budget}} >> r_{\text{max}}$ for some $i \in l_{\text{selected}}$, the remainder rank will be distributed to other selected layers based on their P_{rank}. The calculated r_{i} values are stored in a list r_{selected}. The pseudo code of the proposed adaptive LoRA allocation is summarized in Algorithm 1. Similar to Fed-HeLLo [18], we apply a short warm-start on clients whose resource budgets can support the warm-start configuration, to stabilize optimization and obtain reliable sensitivity signals. After warm-start, HRALoRA strictly enforces each client's estimated layer and rank budgets.

Algorithm 1 Adaptive LoRA rank allocation

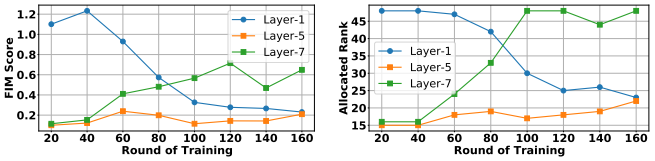
Require: The layer budget l_{budget}, the rank budget r_{budget}

1: Initialize an empty list of FIM scores FIM_{list}, the list of selected layers for LoRA l_{selected} and the corresponding list of LoRA ranks r_{selected}.
2: **for** each layer i in the global model **do**
3: Calculate the FIM score $FIM_{\text{score,i}}$ following Eq. (9)
4: Pushed $FIM_{\text{score,i}}$ to the FIM score list FIM_{list}
5: Obtain l_{selected} by selecting l_{budget} layers with probabilities generated from FIM_{list} following the layer selection in Fed-HeLLo [18]
6: Put the FIM scores of selected layers in a new list FIM_{selected}.
7: Normalize FIM_{selected} to obtain $P_{\text{rank,i}}$ for $i \in l_{\text{selected}}$.
8: Calculate r_{i} following Eq. (10) and push it into r_{selected}.
9: **return** l_{selected} and r_{selected}

An example of the computed FIM scores and the corresponding allocated LoRA ranks for each layer when fine-tuning the ViT model *facebook/deit-small-patch16-224* [14] on the IID CIFAR100 [13] dataset is presented in Fig. 3 (a) and (b). The results show that different layers exhibit varying



(a) Layer FIM score mean and std (b) Layer allocated rank mean and std



(c) FIM score in layer 1, 5, and 7. (d) Allocated rank in layer 1, 5, and 7.

Fig. 3. Layer-wise mean and standard deviation of FIM scores and allocated ranks when training on IID CIFAR100 data with 20 clients for 200 rounds. The big standard deviation in (a) and (b) implies a sensitivity drift during training, as shown in (c) and (d) for layer-1 and 7. Here we use $r_{\text{max}} = 48$.

levels of sensitivity to the dataset, and the allocated LoRA rank adapts accordingly based on the FIM score. Fig. 3 (c) and (d) further illustrate the evolution of FIM scores during training for selected layers. The rank selection mechanism consistently assigns higher LoRA capacity to the layers deemed more important, thereby improving overall fine-tuning performance.

E. Alternating LoRA Training and SVD-based Aggregation

In the vanilla federated learning [1], the server aggregates LoRA matrices and averages them individually. However, with data heterogeneity, a "client-drift" phenomenon [4] is observed, which causes non-stable training and low convergence rate. This happens when the local optimum drifts from the global optimum, and it is more prominent with non-IID data. Specifically, researchers [4], [16], [27] observe that in a non-IID scenario, averaging LoRA matrices individually does not yield averaged LoRA from different clients:

$$W = W_0 + \frac{1}{N}\sum_{i=1}^{N} B_i \times \frac{1}{N}\sum_{i=1}^{N} A_i \neq W_0 + \frac{1}{N}\sum_{i=1}^{N}(B_i \times A_i) \tag{11}$$

where N is the total number of clients. To address this, FFA-LoRA [16] proposes to train and aggregate matrix B only, and FLoRA [27] aggregates the matrix product. Inspired by FFA-LoRA and ADMM, which optimizes variables alternatively and provides strong convergence guarantees in convex optimization [21], HRALoRA trains A and B alternately for local LoRA update. By alternating the training matrix, we avoid the cases where A, as the encoder in LoRA, is projecting the input to a subspace that can not well represent the data.

After each round of federated training, the weights of LoRA modules from heterogeneous clients are aggregated in the server. We zero pad the matrices with a smaller rank for dimension matching [33], and an averaging is applied to the aggregated matrices of the same layer. Note that only one of the LoRA matrices is trained and averaged in each round due

to the alternating training. Inspired by the weight balance in deep learning [40], [41], after the matrix update, we apply an SVD-based aggregation to balance the weight magnitude within each LoRA module. The experiment shows that this approach improves the fine-tuning performance and the SVD-based aggregation for magnitude balancing can be presented mathematically as follows,

$$B_i A_i = USV^T, \quad B_{i+1} = US^{1/2}, \quad A_{i+1} = S^{1/2}V^T \quad (12)$$

where $(.)^T$ denotes the transpose, $(.)^{1/2}$ represents square root operation, and i is the iteration number. SVD-based aggregation ensures that LoRA matrices A and B start with balanced weights after each round of training.

IV. Experiments

A. Experiment Setup

1) Infrastructure: All experiments were run on a virtual machine provisioned with an NVIDIA H100 GPU with 80GB of VRAM, and an Intel XEON GOLD 6542Y CPU with 24 cores and 128 GB of general-purpose memory. The virtual machine runs Ubuntu 22.04.5 LTS. Distributed training is conducted via the Hugging Face Accelerate library [42].

2) Distributed Environment Configuration: We simulate a distributed environment containing one server and 20 clients following Fed-HeLLo [18] and implement the comparison methods described in Section IV-A4 under the same environment. In each round, 5 clients are selected to participate in training. The client performs training with its local data and sends weight updates to the server for global aggregation. To simulate resource heterogeneity, we divide the 20 clients into 3 groups, where each group has different GPU memory and network bandwidths. Within the same group, the resource is assumed to be the same. The available GPU memory is 2, 4, 8 GB, the available network bandwidth for uploading is 7.1, 10.6, 14.2 Mbps respectively and the download speed is 51.1, 63.4, 86.3 Mbps. This resource setting can be found in common devices like budget laptops and some smartphones.

To ensure all clients can complete each round of training in time while transmitting all updated weights in no greater than 1 second per participating client, we run our rank budget estimation algorithm, which gives their rank and layer budgets summarized in Table III. We apply LoRA to both "query" and "value" components in the transformer and evenly split the rank budget between them. Moreover, LoRA for "query" and "value" in the same layer have the same rank following the literature [10], [18]. Thus, the average rank of each LoRA module $r_{\text{avg}} = r_{\text{budget}}/(2 \text{ ("query" and "value")} \times l_{\text{budget}}) = 24$ for all clients. Rank allocation is executed every twenty rounds and for each task, 200 rounds of training are run with a batch size of 32. On average, the rank allocation process, including FIM score computation, requires 0.13 s, accounting for 0.63% of the round time. The SVD-based aggregation introduces an overhead of 0.26 s, corresponding to 1.3% of the round time.

TABLE III
Experimental configuration of different LoRA budgets for resource-heterogeneous client groups.

Group	Clients	Mem.	Upload/download	r_{budget}	l_{budget}
1	6	2 GB	7.1/51.1 Mbps	288	6
2	6	4 GB	10.6/63.4 Mbps	432	9
3	8	8 GB	14.2/86.3 Mbps	576	12

3) Dataset:

- **Image Classification Task:** For the image classification task, we consider the ViT model *facebook/deit-small-patch16-224* [14] pretrained on ImageNet and fine-tuned on the CIFAR100 dataset [13], consisting of 32 by 32 RGB images with 100 different types of classes. This model consists of 12 Transformer blocks and we apply the LoRA modules to the "query" and "value" components for selected layers. For clients whose layer budgets are 12, LoRA modules in all layers are trained.
- **Natural Language Text Classification Task:** For the Natural Language Text Classification task, we employ the pretrained BERT-based model *google/bert_uncased_L-12_H-128_A-2* [43] and fine-tune it on the LEDGAR dataset [44], [45] which contains contract provisions for legal language, classified into 100 classes. Similar to the ViT employed in the image task, this BERT model consists of 12 Transformer blocks and we apply the LoRA to the "query" and "value" components for each transformer.
- **Data Heterogeneity:** We consider multiple variations of data distribution to evaluate the effect of data heterogeneity across clients. Specifically, we run each experiment under three scenarios.
  (i) IID: All clients receive randomly, identically distributed training samples. (ii) Non-IID 20: Each client randomly samples from 20 out of 100 classes, the distribution of the same class among different clients follows a Dirichlet distribution. (iii) Non-IID 10: Each client randomly samples from 10 out of 100 classes.

4) Compared Methods: We compare our methods with several classical federated learning approaches and resource-heterogeneity methods under the same resource constraint. We implement baselines following the original descriptions and fine tune them under our experimental setup. The code will be released alongside our own implementation.

- **FedIT [12]:** The federated learning baseline method combining FedAvg [1] with LoRA. Clients are grouped and assigned the same LoRA layer budget as in Table III. All LoRA module has a rank of 24. LoRA layers are distributed evenly without dynamic allocation. For clients with a layer budget smaller than the total number of transformer blocks, the activated LoRA layers are evenly assigned across the model with the same separation.
- **FFA-LoRA [16]:** Same as FedIT, but matrix A of each LoRA layer is initialized and frozen, while matrix B remains trainable. This method reduces the number of

trainable parameters by 50% compared to FedIT.

- **Low-Rank Kronecker Product (LoKr) [15]:** Same as FedIT, but with all LoRA modules replaced by LoKr $= B \otimes A$, which significantly reduces the trainable parameters. The size of B and A matrices in LoKr are determined by the input and output sizes of the "query" and "value" components in the transformers.
- **Straggler:** Same as FedIT, but all clients follow the budget of the straggler (client with the lowest resource). Thus, the training algorithm does not need to consider the client heterogeneity.
- **Exclusive:** Same as FedIT, but only clients in the highest-resource group are used for model training. Clients from other resource groups, if selected in a training round, remain idle and do not perform local updates.
- **FlexLoRA [25]:** Clients follow the same client grouping and rank budgets as in Table III. But the LoRA rank is uniformly assigned across all layers within each client. For example, a client with a total rank budget of 432 allocates a rank of $432/12/2$ ("query" and "value") $= 18$ to LoRA modules in all layers. SVD is applied for LoRA weights redistribution.
- **HetLoRA [33]:** Clients adopt the same client grouping and rank budget configuration as FlexLoRA, with a uniform LoRA rank across all layers for each client. To accommodate heterogeneous rank budgets, matrix truncation is applied to obtain a lower-rank LoRA for weight redistribution, and aggregation is weighted by the norm of singular values.
- **LEGEND [17]:** This method uses a static rank allocation that prioritizes later layers. For example, for layer budget 9 with rank budget 432, the LoRA modules in the last 9 layers are activated with rank distribution $\{20, 21, 22, 23, 24, 25, 26, 27, 28\}$ for both "query" and "value" components, where 28 is the rank of the LoRA module in the final layer.
- **Fed-HeLLo [18]:** This algorithm re-selects the trainable LoRA layers every 10 training rounds based on the FIM score. All LoRA modules have the same rank of 24.

B. Rank Estimator

To validate our rank estimator's accuracy in memory estimation, we compared estimated memory usage with actual measurements from PyTorch's memory profiler[1], which tracks peak memory allocation during training's forward and backward passes. We use the model in our image task, *facebook/deit-small-patch16-224* with sequence length 197 and hidden dimension 384, FP32 precision, the Adam optimizer, and a batch size of 32 for analysis. Results are collected in Table IV, which shows that our rank estimation can estimate the memory cost of LoRA fine-tuning accurately.

We also evaluate how GPU memory and network bandwidth affect the LoRA rank budget per client. As illustrated by the blue line in Fig. 4, we vary GPU memory to examine memory constraints, without considering the effect of network. For the green line, we vary network speeds. A zero rank means that the memory is not large enough to safely train the model. Compared to the memory of the base model, only a small additional memory is required to enable LoRA training.

[1]https://docs.pytorch.org/docs/main/profiler.html

TABLE IV
MEMORY BREAKDOWN COMPARISON TABLE

Component	Estimated (MB)	Profiled (MB)	Error (%)
Parameters	98.32	98.32 (4.90%)	0.00
Forward Pass	1906.55	1787.53 (89.07%)	6.66
Gradients	14.20	14.20 (0.71%)	0.00
Optimizer States	28.41	28.41 (1.42%)	0.00
Overhead	81.11	78.44 (3.91%)	3.41
Total Peak	2047.48	2006.90 (100.00%)	2.02

Note: In this example, the available GPU memory is 2048 MB with a rank budget of 4776. "Error" measures the percentage difference between the estimated memory usage and the mean profiled memory.

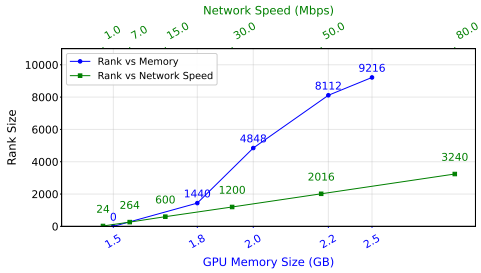


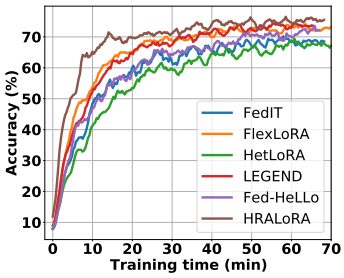
Fig. 4. The blue line represents the rank budget per client as a function of GPU memory. The green line represents the rank budget per client as a function of network speed, assuming the weight transmission time is no greater than 1 second.

C. Image Classification Fine-tuning on CIFAR100

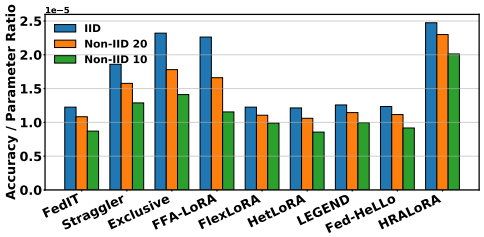
In the image classification task, we consider fine-tuning the pre-trained ViT [14] model, which consists of 12 layers of transformers, on the CIFAR100 dataset. The pre-trained DeiT model is pre-trained on ImageNet [46] and has an accuracy lower than 1% if applied directly to the CIFAR100 dataset. The model accuracy, after fine-tuning, is summarized in Table V where the best Top-1 Accuracy is presented. The highest accuracy is highlighted in bold red, and the second-highest is shown in light red. We additionally report the expected per-round uplink communication volume (MB). For each method, we compute this from the number of parameters updated by a participating client in that round, assuming FP32 (4 bytes/parameter), and aggregate over the clients that actually perform local training in that round (up to 5 selected clients and some methods may idle a subset). Since only parameter updates are transmitted, the upload volume is modeled directly from the number of trainable parameters. For alternating-training methods, only the currently-trained LoRA matrix (A or B) is transmitted in a given round. The smallest and second-smallest expected upload data sizes are highlighted. HRALoRA exceeds the performance of other resource heterogeneous methods with a small expected upload data size.

TABLE V
THE PEFT PERFORMANCE, MEASURED BY ACCURACY ON CIFAR100 USING 20 FEDERATED CLIENTS.

Setting	FedIT	Straggler	Exclusive	LoKr	FFA-LoRA	FlexLoRA	HetLoRA	LEGEND	Fed-HeLLo	HRALoRA
IID Data	84.01%	82.24%	82.11%	66.90%	77.56%	84.00%	83.21%	84.50%	84.67%	**84.81%**
Non-IID 20	74.29%	69.77%	63.00%	54.59%	56.94%	75.83%	72.71%	76.90%	76.49%	**78.83%**
Non-IID 10	59.69%	56.97%	49.94%	43.39%	39.58%	67.86%	58.70%	66.88%	62.88%	**68.99%**
Exp. data size (MB)	6.86	4.42	3.54	**0.31**	3.43	6.86	6.86	6.72	6.86	3.43



(a) Time-to-Accuracy performance.



(b) Ratio of accuracy and number of trainable parameters.

Fig. 5. Testing accuracy and the ratio of accuracy and trainable parameters for different methods. A five-step moving average filter is applied to smooth the curves in (a) for improved visualization.

Fig. 5a presents the time-to-accuracy performance measured in wall-clock time under the non-IID-20 data setting. Wall-clock time is measured as the end-to-end runtime of our single-VM federated emulation (compute + framework overhead) and excludes network transfer time. Network effects are modeled separately from the per-round payload size and client bandwidth constraints. HRALoRA exhibits a faster convergence rate than evaluated baselines. Fig. 5b reports the ratio of achieved accuracy to the total number of trainable parameters across all clients. HRALoRA consistently attains the highest ratio, demonstrating an effective utilization of the parameters.

1) Effect of Non-IID Data Distribution: The advantages of HRALoRA become more evident under heterogeneous client distributions. Under the non-IID 20 condition, HRALoRA reaches 78.83% accuracy, outperforming Fed-HeLLo (76.49%) and substantially exceeding classical baselines such as Straggler (69.77%), Exclusive (63.00%), and FFA-LoRA (56.94%). In the non-IID 10 scenario, each client only samples data from 10 classes, resulting in more extreme data heterogeneity. In this case, the advantage of HRALoRA becomes more pronounced. HRALoRA achieves 68.99% accuracy, again the highest among all methods. The next-best model, LEGEND, trails at 66.88%, and other approaches degrade even further.

2) Larger Scale Environment: To further examine the scalability of PEFT methods in federated settings, we increase the total number of clients to 100 without changing the ratio of group size and client resource, and randomly select 10 clients per round. This setup introduces two additional challenges: (1) substantially higher client diversity, and (2) increased data variance due to sparse participation. The results are presented in Table VI. Across all data regimes, HRALoRA consistently delivers high performance, demonstrating that its intelligent rank allocation and aggregation strategy remains effective even under extreme client heterogeneity and reduced participation frequency. Under the IID configuration, HRALoRA reaches 80.84%, slightly below LEGEND (81.42%) and Fed-HeLLo (81.09%), yet still outperforming other classical baselines. In the non-IID 20 scenario, HRALoRA attains 76.11%, surpassing all other methods. In the most challenging non-IID 10 setting, HRALoRA again achieves the highest accuracy at 70.05%. These results highlight HRALoRA's robustness when clients have extremely fragmented and uneven data distributions, a setting where many PEFT baselines collapse in performance.

D. Text Classification Fine-tuning on LEDGAR

Next we consider the text classification task on LEDGAR dataset [44], [45]. We applied the pre-train BERT model [47] for fine-tuning on LEDGAR. The PEFT result can be found in Table VII. Similar to the result observed in the image classification task, the advantage of HRALoRA becomes pronounced when data heterogeneity is introduced. In the non-IID 20 scenarios, HRALoRA delivers the best micro F1 score among all methods, and this trend continues further under the non-IID 10 condition. Here, HRALoRA achieves a 0.575 F1 score, again the highest among all evaluated methods.

TABLE VII
THE PEFT PERFORMANCE, MEASURED BY MICRO F1 SCORE ON LEDGAR USING 20 FEDERATED CLIENTS.

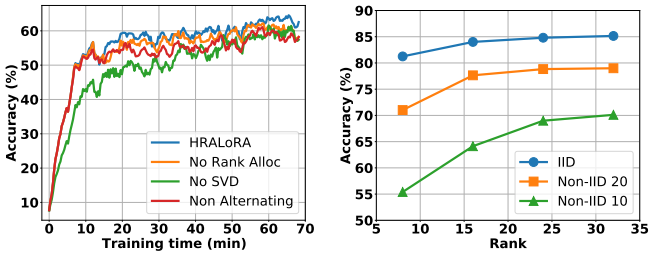
Method	IID Data	Non-IID 20	Non-IID 10
FFA-LoRA	0.685	0.624	0.459
HetLoRA	0.772	0.697	0.541
LEGEND	0.781	0.676	0.549
Fed-HeLLo	**0.789**	0.701	0.527
HRALoRA	0.775	**0.704**	**0.575**

E. Ablation Studies

In this section, we analyze the impact of different components of HRALoRA on overall performance. We first report the test accuracy during CIFAR100 fine-tuning in Fig. 6a. As shown, accuracy increases rapidly during the early stages

TABLE VI
THE PEFT PERFORMANCE, MEASURED BY ACCURACY ON CIFAR100 USING 100 FEDERATED CLIENTS.

Setting	FedIT	Straggler	Exclusive	LoKr	FFA-LoRA	FlexLoRA	HetLoRA	LEGEND	Fed-HeLLo	HRALoRA
IID Data	80.39%	76.10%	78.69%	52.97%	69.35%	79.37%	75.18%	**81.42%**	81.09%	80.84%
Non-IID 20	72.92%	65.47%	69.71%	36.27%	53.70%	71.15%	63.69%	74.98%	74.24%	**76.11%**
Non-IID 10	63.10%	51.84%	58.91%	24.66%	36.61%	60.01%	48.22%	66.74%	66.46%	**70.05%**



(a) Time-to-Accuracy performance. (b) Accuracy with varying rank.

Fig. 6. Testing accuracy during training and with different average rank per LoRA module when fine-tuning CIFAR100. A five-step moving average filter is applied to smooth the curves in (a) for improved visualization.

TABLE VIII
EFFECT OF RANK ALLOCATION FREQUENCY UNDER THE NON-IID 10 SETTING. THE DEFAULT 20-ROUND SETTING ACHIEVES AN ACCURACY OF 68.99% AND AN F1 SCORE OF 0.575 ON CIFAR100 AND LEDGAR FINE-TUNING.

Frequency (rounds)	1	10	20	30
Accuracy Difference	+ 3.71%	+ 1.52%	Ref.	- 0.15%
F1 Score Difference	+ 0.0360	+ 0.0201	Ref.	- 0.0021

of training and gradually converges. Moreover, without SVD-based aggregation for balancing the LoRA matrices, the convergence rate drops significantly. We further examine the effect of average LoRA rank in Fig. 6b. The results show that model performance improves as the average LoRA rank per module increases. Beyond 24, further increasing the LoRA rank leads to only marginal performance gains, indicating that an average rank of 24 provides sufficient representational capacity to capture the majority of task-relevant features.

In addition, we examine the effect of rank allocation frequency under the non-IID 10 setting on CIFAR100 and LEDGAR datasets. The results are documented in Table VIII. These results show that performance generally benefits from more frequent rank allocation. However, the gain is not linear. The impact is most pronounced at high allocation frequencies, and the performance becomes less sensitive once the allocation interval approaches the default rank allocation frequency of every 20 rounds.

To further understand the contribution of each component in our approach, we remove one module at a time. Table IX summarizes the performance degradation, when fine-tuning CIFAR100 and LEDGAR datasets with non-IID 10 data distribution. The results demonstrate that each component plays a meaningful role in achieving the final performance. Removing the rank-selection mechanism leads to a 3.02% accuracy drop and a 0.0048 decrease in F1 score, confirming

TABLE IX
ABLATION STUDY SHOWING ACCURACY AND F1 SCORE DROP FOR DIFFERENT COMPONENT REMOVALS WITH NON-IID 10 DATA DISTRIBUTION ON CIFAR100 AND LEDGAR FINE-TUNING.

	No-Rank Allocation	No-Alternating	No-SVD
Accuracy Drop	3.02%	4.52%	1.12%
F1 Score Drop	0.0048	0.0173	0.0182

the importance of adaptively choosing layer-wise ranks instead of relying on heuristic allocations. Eliminating the alternating training results in the largest accuracy degradation of 4.52% and 0.0173 decline in F1 score, suggesting that alternating the optimization is critical for enhancing system robustness under non-IID data. Without alternating training, the upload data size is doubled, reducing communication efficiency. Finally, removing SVD-based aggregation results in a smaller yet still noticeable degradation in performance. Nevertheless, Fig. 6a illustrates that SVD-based aggregation mechanism plays an important role in improving the training convergence rate.

V. CONCLUSION

In this paper, we propose HRALoRA, a novel LoRA-based federated fine-tuning framework designed for parameter-efficient learning under heterogeneous client resources. We explicitly model memory and communication bandwidth constraints to characterize feasible LoRA ranks and develop a rank budget estimator. Given the estimated rank budget, we introduce a Fisher Information Matrix (FIM) score-based rank allocation strategy that adaptively distributes the LoRA rank across layers according to their importance. To further improve robustness under non-IID data distributions and reduce communication overhead, we design an alternating training scheme that selectively activates LoRA modules during local updates. In addition, we propose an SVD-based aggregation method to effectively fuse heterogeneous LoRA updates across clients. Extensive experiments on both image and language fine-tuning tasks, under IID and non-IID settings, demonstrate the effectiveness and efficiency of HRALoRA.

ACKNOWLEDGMENT

The authors acknowledge utilizing AI-assisted tools, including ChatGPT, Gemini, and Grammarly, for editing, grammar improvement, and document preparation. All conceptual input, system design decisions, experimental work, and technical writing were created and verified by the research team.

REFERENCES

[1] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2017.

[2] Chen Zhang, Yu Xie, Hang Bai, Bin Yu, Weihong Li, and Yuan Gao, "A survey on federated learning," *Knowledge-Based Systems*, 2021.

[3] Jie Wen, Zhixia Zhang, Yang Lan, Zhihua Cui, Jianghui Cai, and Wensheng Zhang, "A survey on federated learning: challenges and applications," *International Journal of Machine Learning and Cybernetics (IJMLC)*, 2023.

[4] Sai Praneeth Karimireddy, Satyen Kale, Mehryar Mohri, Sashank Reddi, Sebastian Stich, and Ananda Theertha Suresh, "Scaffold: Stochastic controlled averaging for federated learning," in *International Conference on Machine Learning (ICML)*, 2020.

[5] Brian Lester, Rami Al-Rfou, and Noah Constant, "The power of scale for parameter-efficient prompt tuning," in *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2021.

[6] Zeyu Han, Chao Gao, Jinyang Liu, Jeff Zhang, and Sai Qian Zhang, "Parameter-efficient fine-tuning for large models: A comprehensive survey," *Transactions on Machine Learning Research (TMLR)*, 2024.

[7] Elad Ben-Zaken, Shauli Ravfogel, and Yoav Goldberg, "Bitfit: Simple parameter-efficient fine-tuning for transformer-based masked language-models," in *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022.

[8] Samuel Horvath, Stefanos Laskaridis, Mario Almeida, Ilias Leontiadis, Stylianos Venieris, and Nicholas Lane, "Fjord: Fair and accurate federated learning under heterogeneous targets with ordered dropout," *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[9] Samyadeep Basu, Shell Hu, Daniela Massiceti, and Soheil Feizi, "Strong baselines for parameter-efficient few-shot fine-tuning," in *AAAI Conference on Artificial Intelligence (AAAI)*, 2024.

[10] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al., "Lora: Low-rank adaptation of large language models," in *International Conference on Learning Representations (ICLR)*, 2022.

[11] Zhuo Zhang, Yuanhang Yang, Yong Dai, Qifan Wang, Yue Yu, Lizhen Qu, and Zenglin Xu, "Fedpetuning: When federated learning meets the parameter-efficient tuning methods of pre-trained language models," in *Annual Meeting of the Association of Computational Linguistics (ACL)*, 2023.

[12] Jianyi Zhang, Saeed Vahidian, Martin Kuo, Chunyuan Li, Ruiyi Zhang, Tong Yu, Guoyin Wang, and Yiran Chen, "Towards building the federatedgpt: Federated instruction tuning," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2024.

[13] Alex Krizhevsky and Geoffrey Hinton, "Learning multiple layers of features from tiny images," 2009.

[14] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou, "Training data-efficient image transformers & distillation through attention," in *International Conference on Machine Learning (ICML)*, 2021.

[15] Ali Edalati, Marzieh Tahaei, Ivan Kobyzev, Vahid Partovi Nia, James J Clark, and Mehdi Rezagholizadeh, "Krona: Parameter-efficient tuning with kronecker adapter," in *Enhancing LLM Performance: Efficacy, Fine-Tuning, and Inference Techniques*. Springer, 2025.

[16] Youbang Sun, Zitao Li, Yaliang Li, and Bolin Ding, "Improving lora in privacy-preserving federated learning," in *International Conference on Learning Representations (ICLR)*, 2024.

[17] Jun Liu, Yunming Liao, Hongli Xu, Yang Xu, Jianchun Liu, and Chen Qian, "Adaptive parameter-efficient federated fine-tuning on heterogeneous devices," *IEEE Transactions on Mobile Computing (IEEE TMC)*, 2025.

[18] Zikai Zhang, Ping Liu, Jiahao Xu, and Rui Hu, "Fed-hello: Efficient federated foundation model fine-tuning with heterogeneous lora allocation," *IEEE Transactions on Neural Networks and Learning Systems (IEEE TNNLS)*, 2025.

[19] David J Weiss and G Gage Kingsbury, "Application of computerized adaptive testing to educational problems," *Journal of Educational Measurement (JEM)*, 1984.

[20] Alexander Soen and Ke Sun, "Trade-offs of diagonal fisher information matrix estimators," *Advances in Neural Information Processing Systems (NeruIPS)*, 2024.

[21] Robert Nishihara, Laurent Lessard, Ben Recht, Andrew Packard, and Michael Jordan, "A general analysis of the convergence of admm," in *International Conference on Machine Learning (ICML)*. PMLR, 2015.

[22] Dongqi Cai, Yaozong Wu, Shangguang Wang, Felix Xiaozhu Lin, and Mengwei Xu, "Efficient federated learning for modern nlp," in *International Conference on Mobile Computing and Networking (MobiCom)*, 2023.

[23] Han Zhou, Xingchen Wan, Ivan Vulić, and Anna Korhonen, "Autopeft: Automatic configuration search for parameter-efficient fine-tuning," *Transactions of the Association for Computational Linguistics (TACL)*, 2024.

[24] Yaqing Wang, Sahaj Agarwal, Subhabrata Mukherjee, Xiaodong Liu, Jing Gao, Ahmed Hassan, and Jianfeng Gao, "Adamix: Mixture-of-adaptations for parameter-efficient model tuning," in *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2022.

[25] Jiamu Bai, Daoyuan Chen, Bingchen Qian, Liuyi Yao, and Yaliang Li, "Federated fine-tuning of large language models under heterogeneous tasks and client resources," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[26] Sara Babakniya, Ahmed Roushdy Elkordy, Yahya H Ezzeldin, Qingfeng Liu, Kee-Bong Song, MOSTAFA EL-Khamy, and Salman Avestimehr, "Slora: Federated parameter efficient fine-tuning of language models," in *NeurIPS Workshop on Federated Learning in the Age of Foundation Models*, 2023.

[27] Ziyao Wang, Zheyu Shen, Yexiao He, Guoheng Sun, Hongyi Wang, Lingjuan Lyu, and Ang Li, "FLoRA: Federated fine-tuning large language models with heterogeneous low-rank adaptations," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[28] Feijie Wu, Zitao Li, Yaliang Li, Bolin Ding, and Jing Gao, "Fedbiot: Llm local fine-tuning in federated learning without full model," in *ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2024.

[29] Fan Lai, Xiangfeng Zhu, Harsha V. Madhyastha, and Mosharaf Chowdhury, "Oort: Efficient federated learning via guided participant selection," in *Symposium on Operating Systems Design and Implementation (OSDI)*, 2021.

[30] Enmao Diao, Jie Ding, and Vahid Tarokh, "Heterofl: Computation and communication efficient federated learning for heterogeneous clients," in *International Conference on Learning Representations (ICLR)*, 2021.

[31] Minjae Kim, Sangyoon Yu, Suhyun Kim, and Soo-Mook Moon, "Depthfl: Depthwise federated learning for heterogeneous clients," in *International Conference on Learning Representations (ICLR)*, 2022.

[32] David Forsyth, *Applied machine learning*, Springer, 2019.

[33] Yae Jee Cho, Luyang Liu, Zheng Xu, Aldi Fahrezi, and Gauri Joshi, "Heterogeneous LoRA for federated fine-tuning of on-device foundation models," in *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.

[34] Xiaoyu Cao, Minghong Fang, Jia Liu, and Neil Zhenqiang Gong, "Fltrust: Byzantine-robust federated learning via trust bootstrapping," in *ISOC Network and Distributed System Security Symposium (NDSS)*, 2021.

[35] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Ł ukasz Kaiser, and Illia Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds. 2017, vol. 30, Curran Associates, Inc.

[36] Ilya Loshchilov and Frank Hutter, "Decoupled weight decay regularization," 2019.

[37] Herbert Robbins and Sutton Monro, "A stochastic approximation method," *The Annals of Mathematical Statistics*, vol. 22, no. 3, pp. 400–407, 1951.

[38] Junzhou Xu, Boyu Diao, Chengqiang Qi, Shaobo Zhao, Ruisheng Wang, and Yongjun Xu, "A sensitivity-driven expert allocation method in lora-moe for efficient fine-tuning," in *IEEE International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, 2025.

[39] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *The Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 2019.

[40] Shibani Santurkar, Dimitris Tsipras, Andrew Ilyas, and Aleksander Madry, "How does batch normalization help optimization?," *Advances in neural information processing systems (NeruIPS)*, 2018.

[41] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton, "Layer normalization," *arXiv preprint arXiv:1607.06450*, 2016.
[42] Sylvain Gugger, Lysandre Debut, Thomas Wolf, Philipp Schmid, Zachary Mueller, Sourab Mangrulkar, Marc Sun, and Benjamin Bossan, "Accelerate: Training and inference at scale made simple, efficient and adaptable.," https://github.com/huggingface/accelerate, 2022.
[43] Iulia Turc, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova, "Well-read students learn better: On the importance of pre-training compact models," *arXiv preprint arXiv:1908.08962v2*, 2019.
[44] Don Tuggener, Pius Von Däniken, Thomas Peetz, and Mark Cieliebak, "Ledgar: a large-scale multi-label corpus for text classification of legal provisions in contracts," in *International Conference on Language Resources and Evaluation (LREC)*, 2020.
[45] Ilias Chalkidis, Abhik Jana, Dirk Hartung, Michael Bommarito, Ion Androutsopoulos, Daniel Katz, and Nikolaos Aletras, "Lexglue: A benchmark dataset for legal language understanding in english," in *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022.
[46] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2009.
[47] Iulia Turc, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova, "Well-read students learn better: On the importance of pre-training compact models," *arXiv preprint arXiv:1908.08962*, 2019.